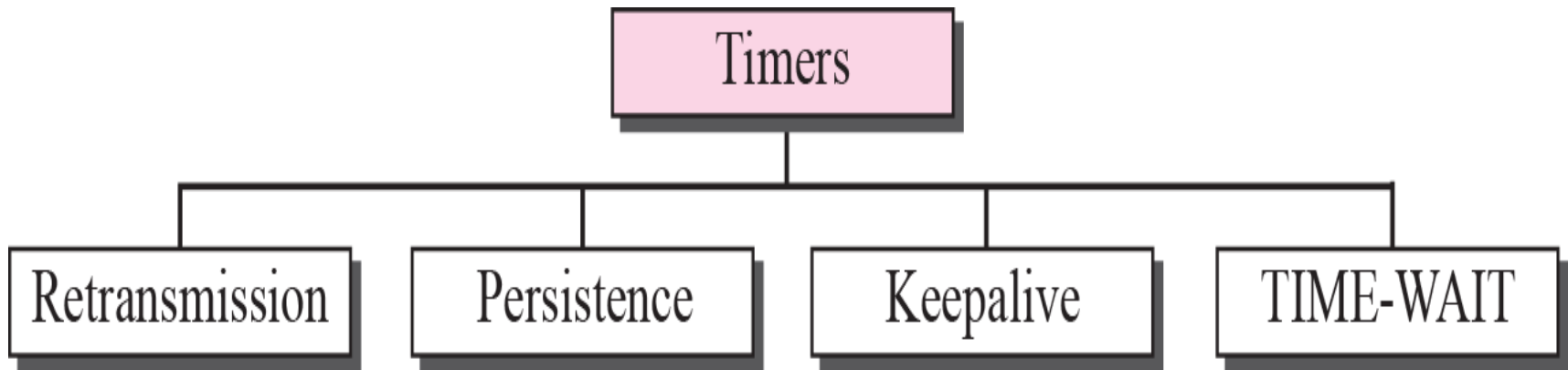


**Transmission Control Protocol
(TCP)
Timers
&
Header Option**

TCP TIMERS

To perform its operation smoothly, most TCP implementations use at least four timers



In TCP, there can be only one RTT measurement in progress at any time.

ReTransmission Timeout (RTO)

Calculations

ReTransmission Timeout (RTO)

1. When TCP sends the segment in front of the sending queue, it starts the timer.
2. When the timer expires, TCP resends the first segment in front of the queue, and restarts the timer.
3. When a segment (or segments) are cumulatively acknowledged, the segment (or segments) are purged from the queue.
4. If the queue is empty, TCP stops the timer; otherwise, TCP restarts the timer.

the segments and their ACKs do not have a 1-1 relationship

ReTransmission Timeout (RTO)

**To calculate retransmission time out (RTO)
we need to calculate the RTT**

Measured RTT (RTT_m) : how long it takes to send a segment and receives an acknowledgement

Smoothened RTT (RTT_s) : weighted average of RTT_m and previous RTTs

RTT Deviation (RTT_d) : deviation in RTT over various transmission

Calculation of RTO

▶ Smoothed RTT: RTT_S

- ▶ Original → No value
- ▶ After 1st measurement → $RTT_S = RTT_M$
- ▶ 2nd ... → $RTT_S = (1-\alpha)*RTT_S + \alpha*RTT_M$

▶ RTT Deviation : RTT_D

- ▶ Original → No value
- ▶ After 1st measurement → $RTT_D = 0.5*RTT_M$
- ▶ 2nd ... → $RTT_D = (1-\beta)*RTT_D + \beta*|RTT_S - RTT_M|$

In TCP, there can be only one RTT measurement in progress at any time.

Calculation of RTO... contd

▶ Retransmission Timeout (RTO)

- ▶ **Original → Initial value**
- ▶ **After any measurement**
→ $RTO = RTT_S + 4RTT_D$

▶ Default values

- ▶ $\alpha = 1/8$
- ▶ $\beta = 1/4$

▶ Smoothed RTT: RTT_S

- ▶ Original → No value
- ▶ After 1st measurement → $RTT_S = RTT_M$
- ▶ 2nd ... → $RTT_S = (1-\alpha)*RTT_S + \alpha*RTT_M$

▶ RTT Deviation : RTT_D

- ▶ Original → No value
- ▶ After 1st measurement → $RTT_D = 0.5*RTT_M$
- ▶ 2nd ... → $RTT_D = (1-\beta)*RTT_D + \beta*|RTT_S - RTT_M|$

Default values

$$\alpha = 1/8$$

$$\beta = 1/4$$

▶ Retransmission Timeout (RTO)

Original → Initial value

After any measurement

$$\rightarrow RTO = RTT_S + 4RTT_D$$

Example

The figure shows the connection establishment and part of the data transfer phases.

RTO = 6

Consider hypothetical example. shows part of a connection.

➤ the value of variables at this moment:

1. When the SYN segment is sent,
 - ❑ there is no value for RTT_M , RTT_S , or RTT_D .
 - ❑ Assume the value of RTO is set to 6.00 seconds.

2. When the SYN+ACK segment arrives
 RTT_M is measured and is equal to 1.5 seconds.

After 1st measurement

→ $RTT_S = RTT_M$

→ $RTT_D = 0.5 \times RTT_M$

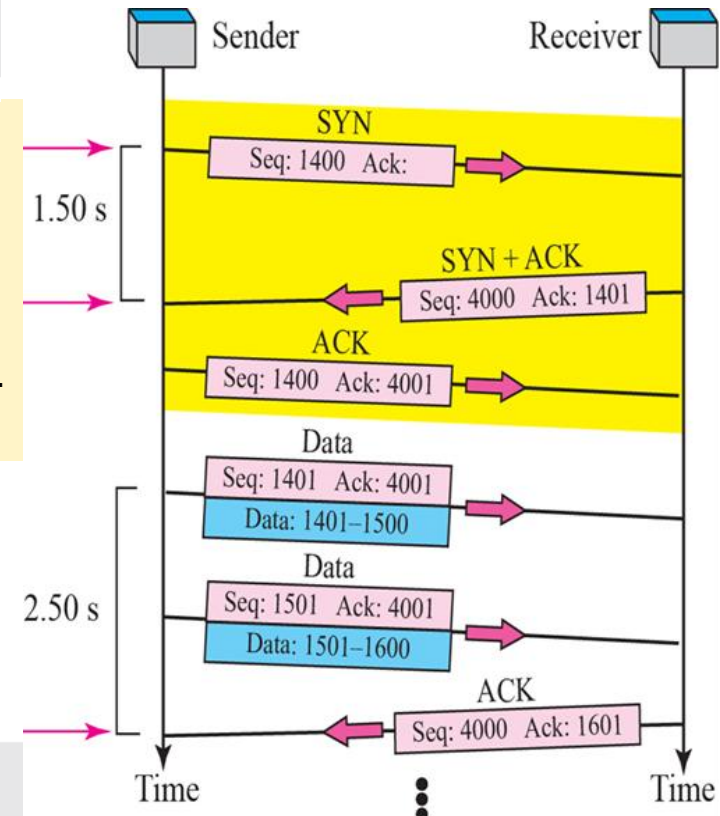
$RTO = RTT_S + 4RTT_D$

$$RTT_M = 1.5$$

$$RTT_S = 1.5$$

$$RTT_D = (1.5) / 2 = 0.75$$

$$RTO = 1.5 + 4 \times 0.75 = 4.5$$



Example

3. When the first data segment is sent, a new RTT measurement starts.

No RTT measurement starts for the second data segment because a measurement is already in progress.

The arrival of the last ACK segment is used to calculate the next value of RTT_M .

Although the last ACK segment acknowledges both data segments (cumulative), its arrival finalizes the value of RTT_M for the first segment.

The values of these variables are now as shown below.

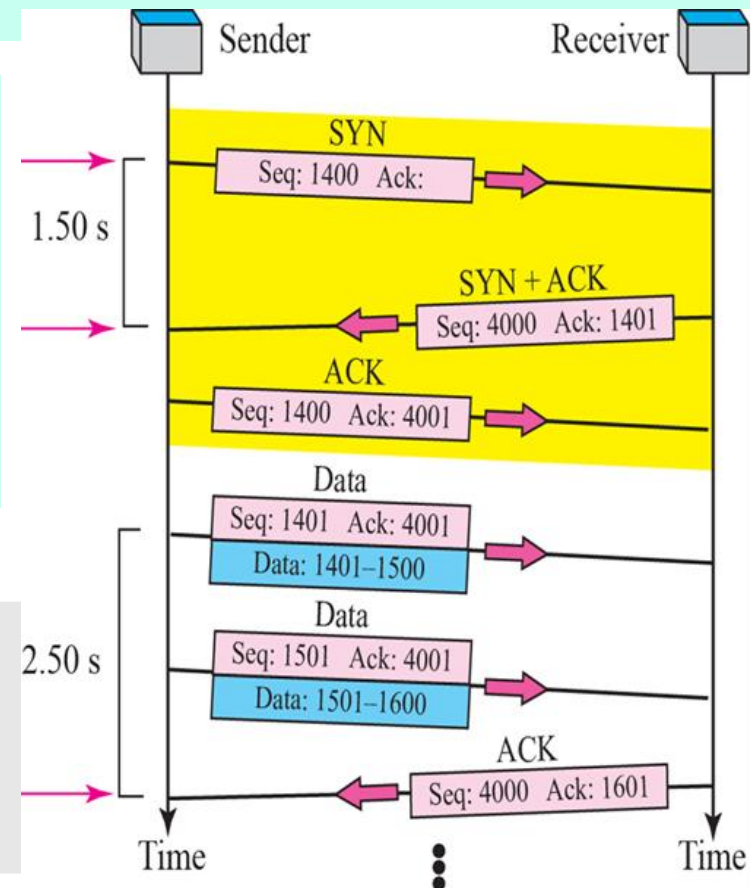
$$\begin{aligned} \rightarrow RTT_S &= (1-\alpha) \cdot RTT_S + \alpha \cdot RTT_M \\ \rightarrow RTT_D &= (1-\beta) \cdot RTT_D + \beta \cdot |RTT_S - RTT_M| \end{aligned}$$

$$RTT_M = 2.5$$

$$RTT_S = 7/8 \times 1.5 + (1/8) \times 2.5 = 1.625$$

$$RTT_D = 3/4 (7.5) + (1/4) \times |1.625 - 2.5| = 0.78$$

$$RTO = 1.625 + 4 \times 0.78 = 4.74$$



The figure shows the connection establishment and part of the data transfer phases.

RTO is keep changing

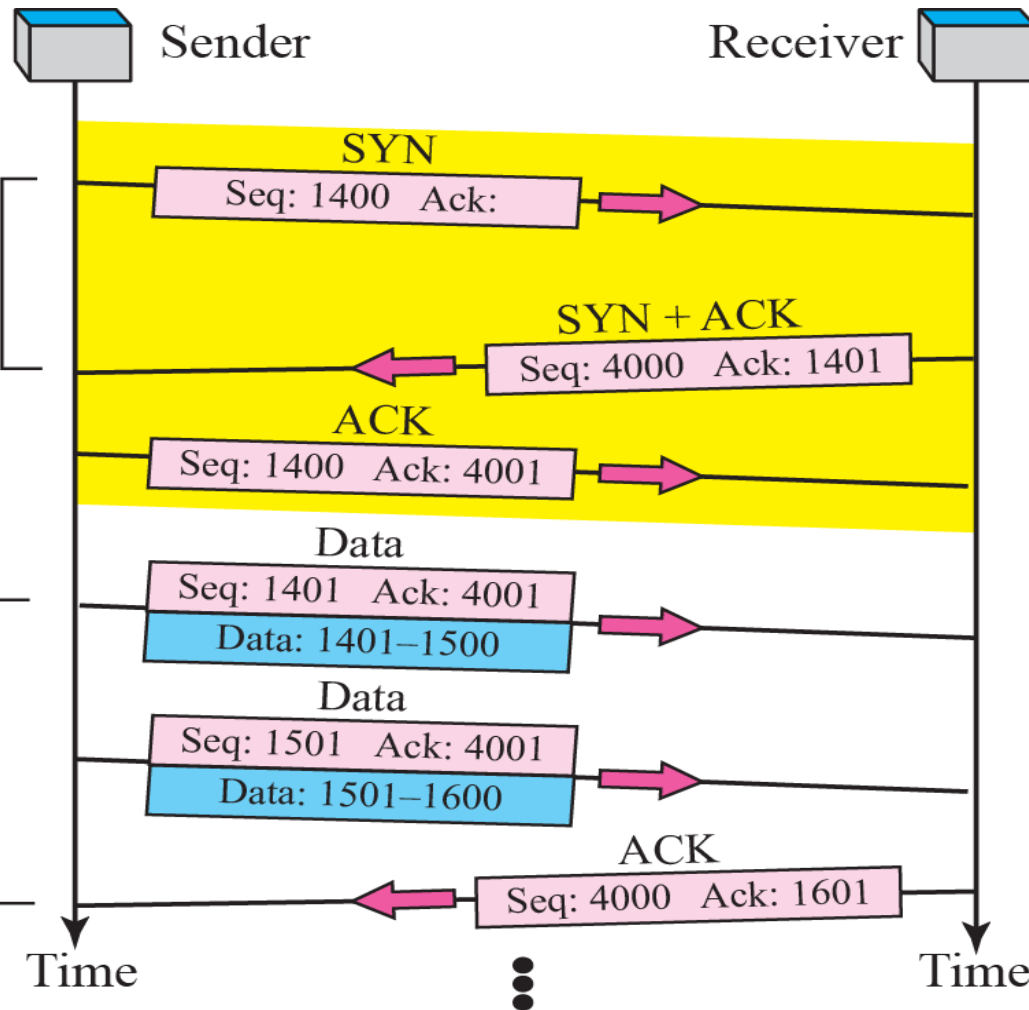
$RTT_M =$ $RTT_S =$
 $RTT_D =$ **RTO = 6.00**

$RTT_M = 1.5$ $RTT_S = 1.50$
 $RTT_D = 0.75$ **RTO = 4.50**

$RTT_M = 2.50$ $RTT_S = 1.625$
 $RTT_D = 0.78$ **RTO = 4.74**

1.50 s

2.50 s



Karn's Algorithm

- Do not consider the round-trip time of a retransmitted segment in the calculation of RTTs.
- Do not update the value of RTTs until you send a segment and receive an acknowledgment without the need for retransmission.

TCP does not consider the RTT of a retransmitted segment in its calculation of a new RTO.

Exponential Backoff

What is the value of RTO if a retransmission occurs?.

- The value of RTO is doubled for each retransmission. So if the segment is retransmitted once, the value is two times the RTO.
- If it transmitted twice, the value is four times the RTO, and so on

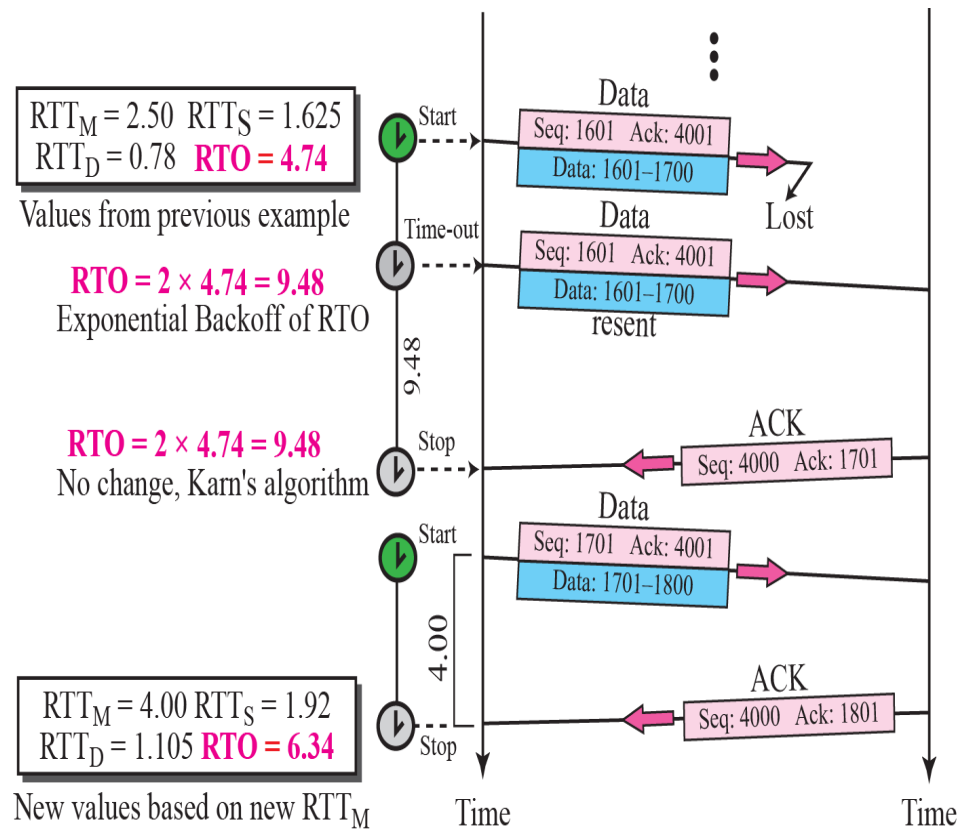
Example

Extending the previous example.

There is retransmission and **Karn's algorithm** is applied.

The first segment in the figure is sent, but lost.

The RTO timer expires after 4.74 seconds.



The segment is retransmitted and the timer is set to 9.48, twice the previous value of RTO. This time an ACK is received before the time-out.

We wait until we send a new segment and receive the ACK for it before recalculating the RTO (Karn's algorithm).

Example

Retransmission and Karn's algorithm

$RTT_M = 2.50$ $RTT_S = 1.625$
 $RTT_D = 0.78$ **$RTO = 4.74$**

Values from previous example

$RTO = 2 \times 4.74 = 9.48$

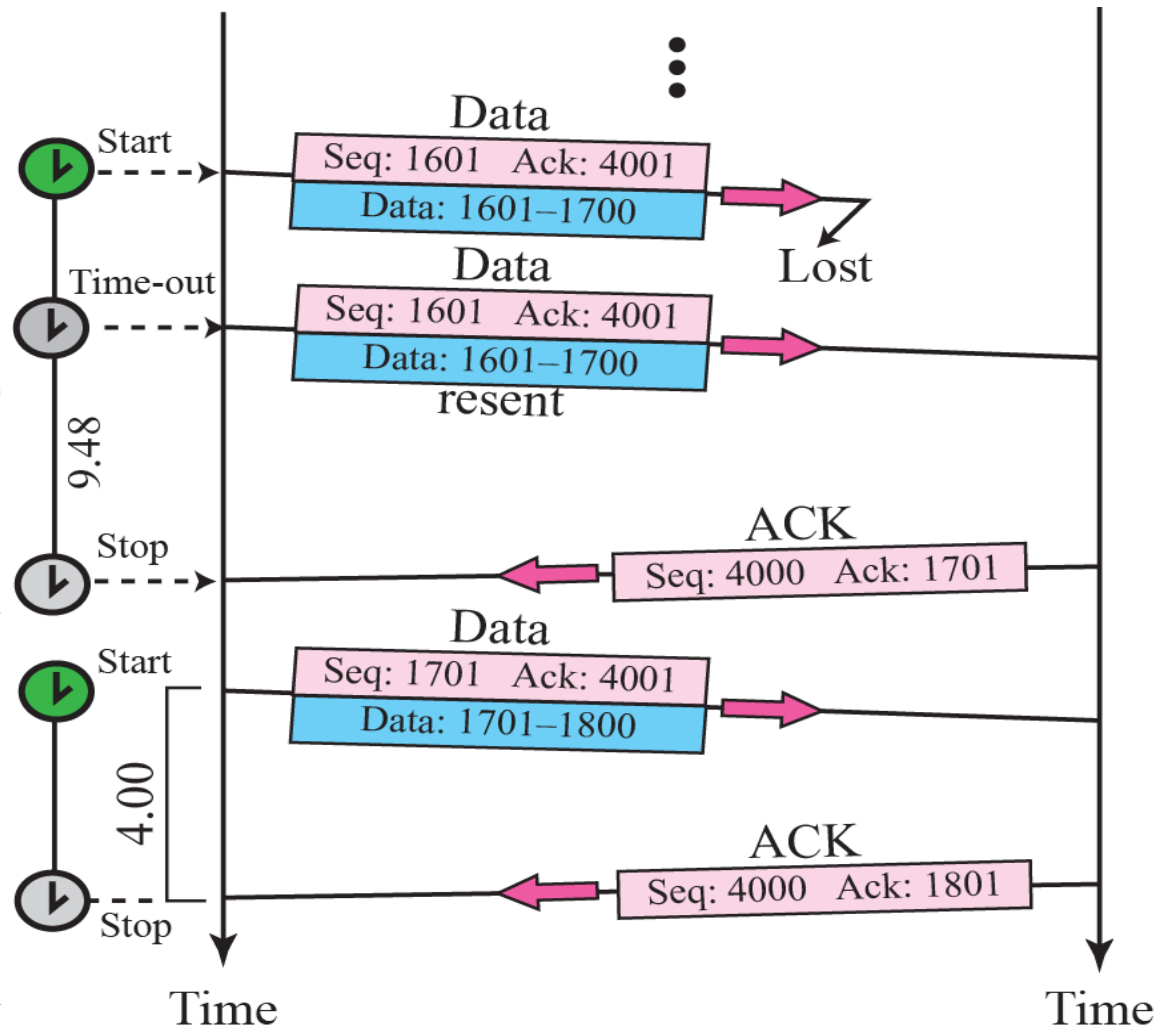
Exponential Backoff of RTO

$RTO = 2 \times 4.74 = 9.48$

No change, Karn's algorithm

$RTT_M = 4.00$ $RTT_S = 1.92$
 $RTT_D = 1.105$ **$RTO = 6.34$**

New values based on new RTT_M



Persistent Timer

Used to deal with zero-window –size advertisement

If ACK segment containing noz-zero window advt got lost.

Since there is no retransmission timer for ACK segment, this may create a deadlock where both ends continue to wait for each other

Resolved with persistent Timer

Whenever *a sender receives a zero-size-window advt, it starts its persistent timer*

On this timer expiry, it sends a special segment called **probe**

- The **value of this timer is set to value of retransmission timer**
- If the acknowledgement is not rcvd, then another probe is sent with a double value.
- It keeps on doubling until threshold (usually 60 s) After that it **send one probe after every 60 s**

Keep Alive Timer

To prevent long idle connection between two TCPs

Server equips with keepalive timer

- Each time server hears from a client, it reset its timer.
- Time-out is usually 2 hours (120 mins).
- If nothing hears from client within this period, it sends a probe segment.
- If no response after **sending 10 probes, after 75 secs apart**, connection will be terminated

Time Wait Timer

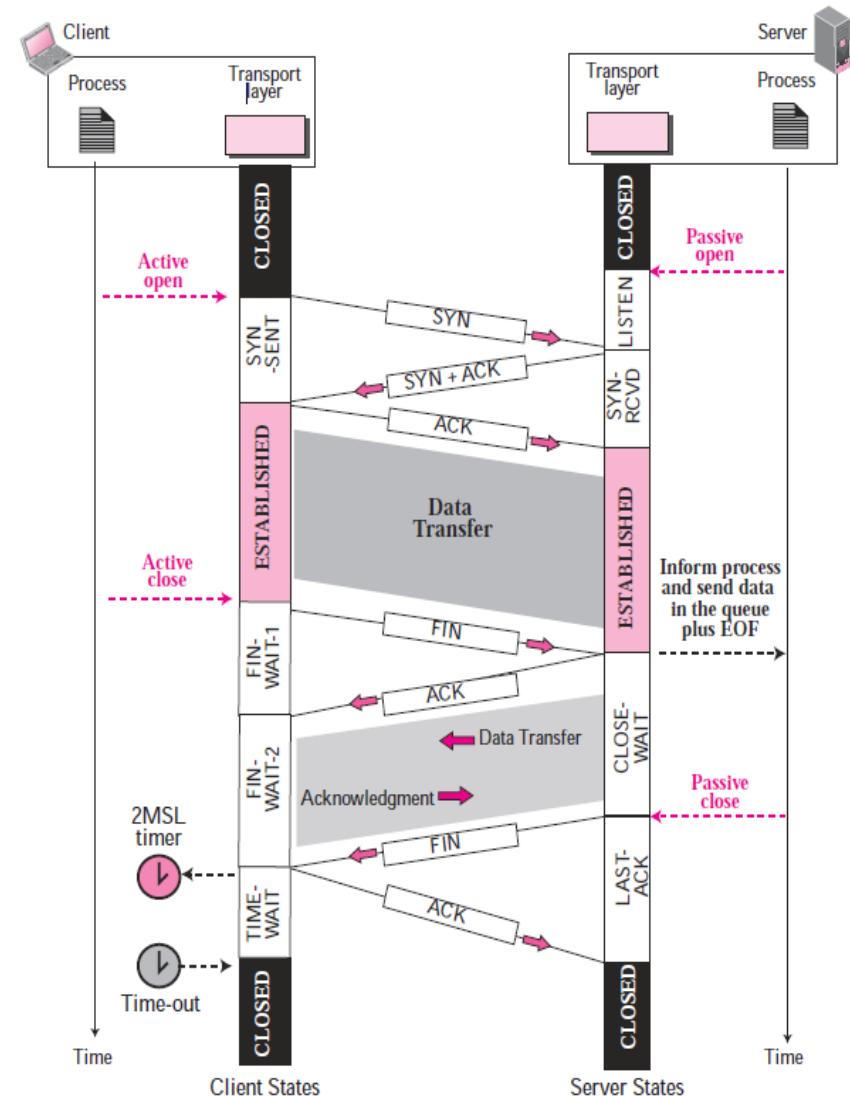
Used during connection termination

Keep hold the resources for $2 * \text{Life time}$
(Max segment life/ Life Time)

Prevent undelivered segment after the connection termination to be delivered to a wrong process if the same port number is assigned to a new process

Maximum segment lifetime (MSL)
is the time a TCP segment can exist in the internetwork system.

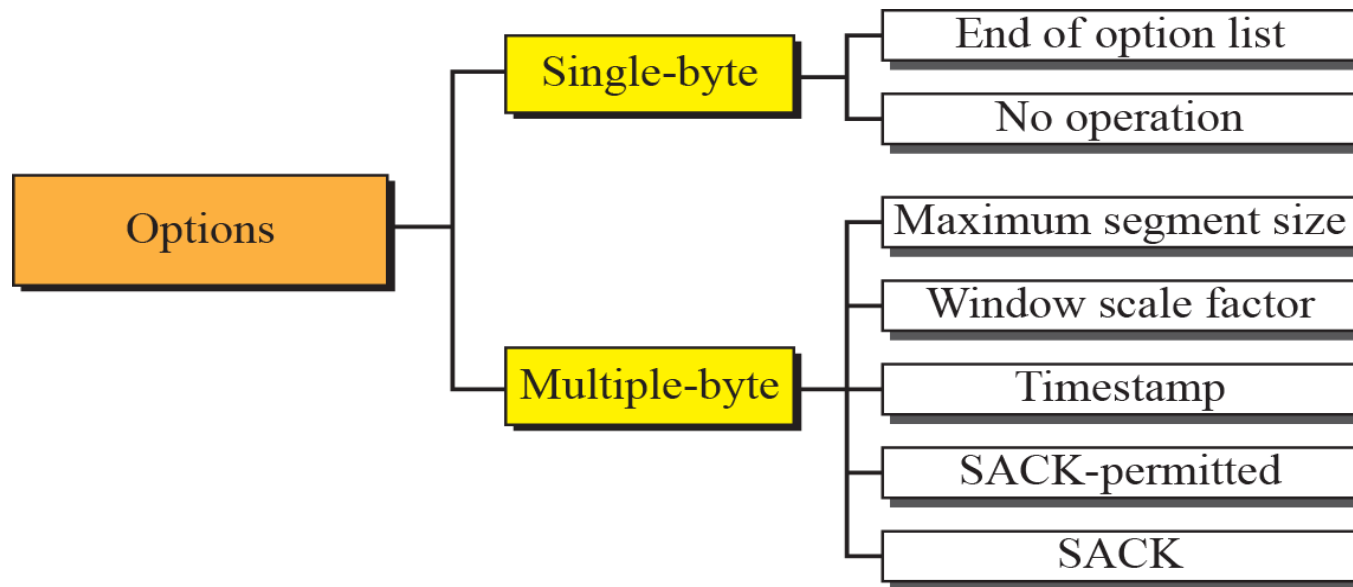
It was defined to be **2 minutes**



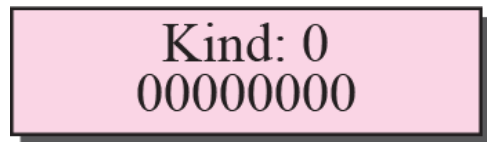
OPTIONS

The TCP header can have up to 40 bytes of optional information.

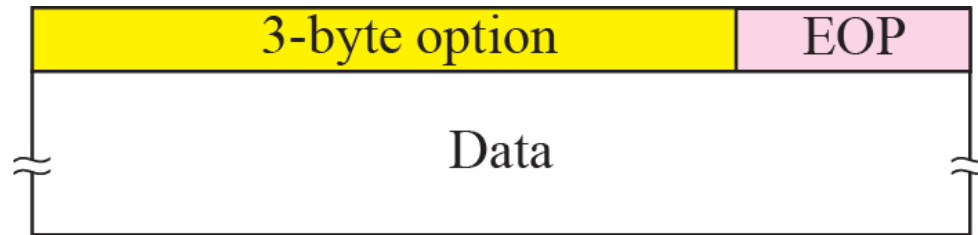
Options convey additional information to the destination or align other options.



End-of-option option



a. End of option list



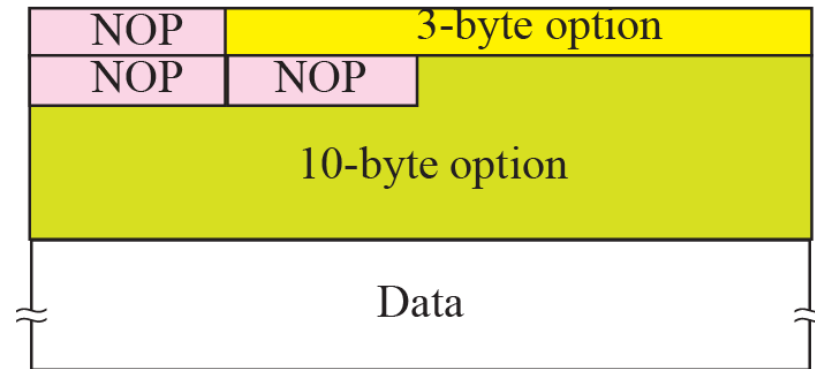
b. Used for padding

EOP can be used only once.



a. No operation option

No-operation option



b. Used to align beginning of an option

NOP can be used more than once.

Maximum-segment-size option

Code: 2 00000010	Length: 4 00000100	Maximum segment size
1 byte	1 byte	2 bytes

Kind: 2

Value(bytes) : 2 bytes

Default value is 536 bytes

The value of MSS is determined during connection establishment and does not change during the connection.

Window-scale-factor option (wsf)

Code: 3 00000011	Length: 3 00000011	Scale factor
1 byte	1 byte	1 byte

Kind: 3

Scale factor : 1 bytes

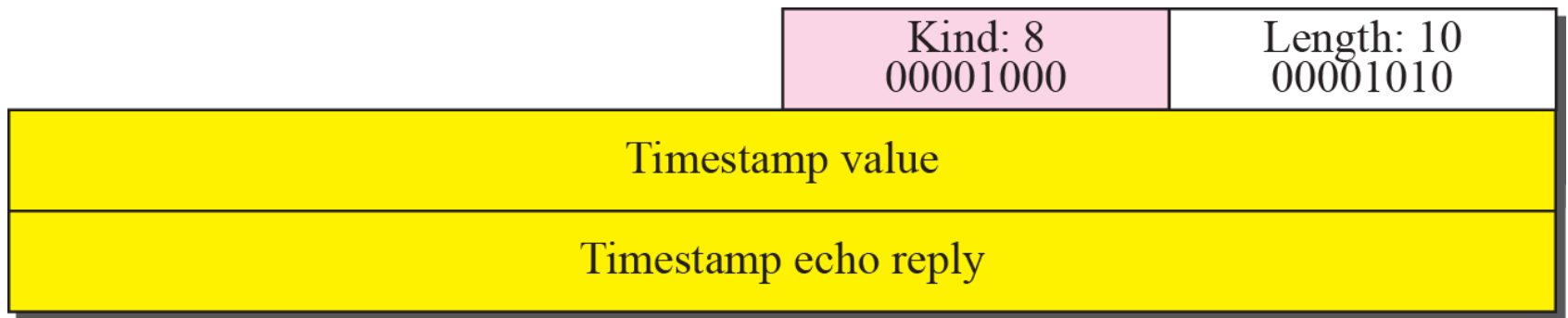
New window size = window size in header * 2^{wsf}

Though the scale factor could be as large a 255, the largest value allowed by TCP/IP is 14

(this is to avoid window size getting larger than sequence numbers)

The value of the window scale factor can be determined only during connection establishment; it does not change during the connection.

Timestamp option

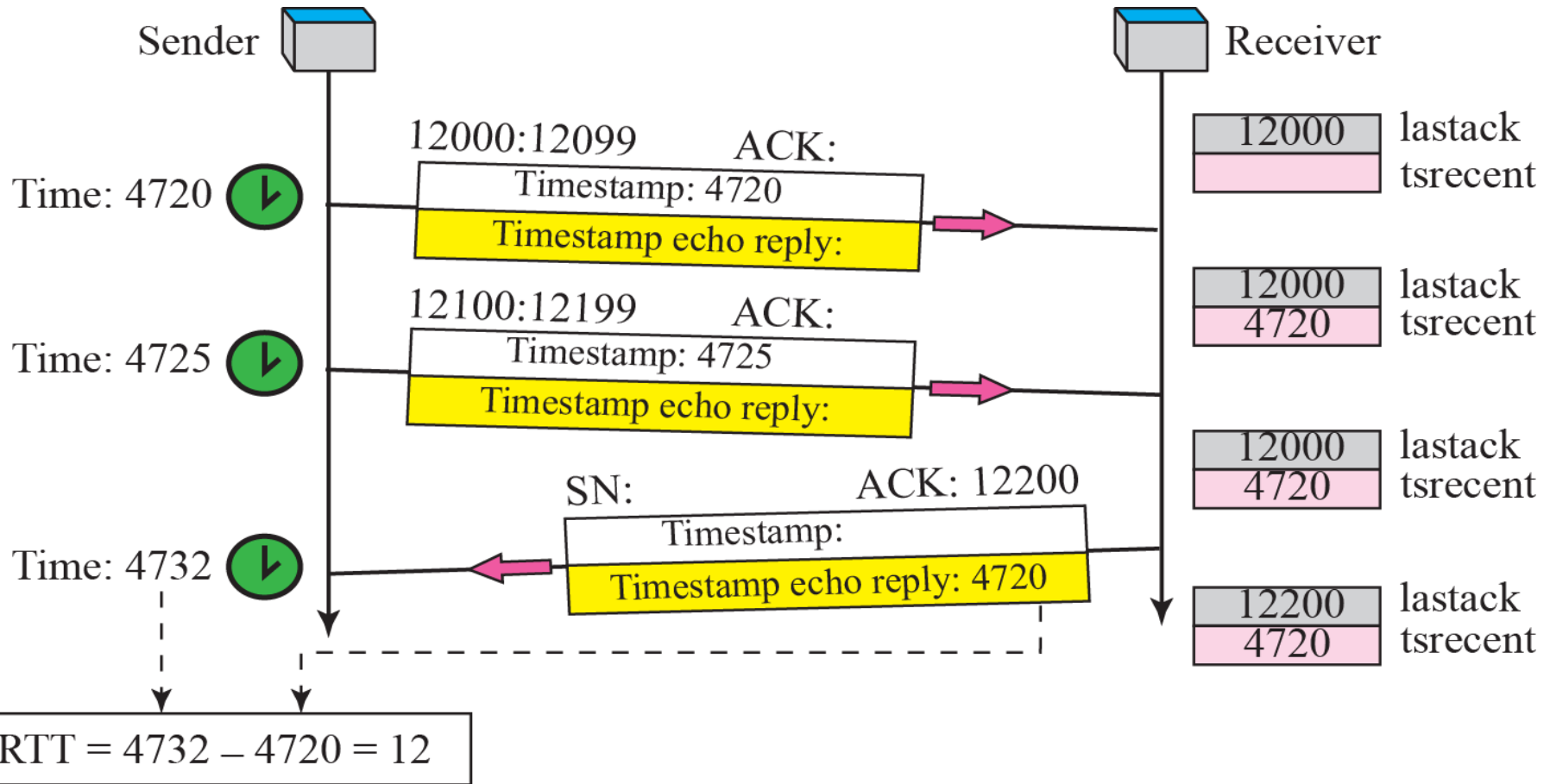


One application of the timestamp option is the calculation of round-trip time (RTT).

Other application of the timestamp option is prevention of Wrap Around Sequences

Example

an example that calculates the round-trip time for one end.



Protections Against Wrapped Sequence Numbers (PAWS)

Wrapping around of sequences is possible in high speed connections , where a same sequence can be used during the lifetime of same connection. This may arise a conflict if two segment with same sequence numbers arrive at same time

Solution

Include a timestamp in the identification of a segment, Identity of a segment is defined as combination of

Timestamp + Sequence Number

Ex;

Two segments 400 : 12001 and 700 : 12001

can be easily distinguished as first sent at time 400 and second sent at time 700

timestamp option is also used for prevention of Wrap Around Sequences



SACK

Kind: 4

Length: 2

SACK-permitted option

Kind: 5

Length

Left edge of 1st Block

Right edge of 1st Block

⋮

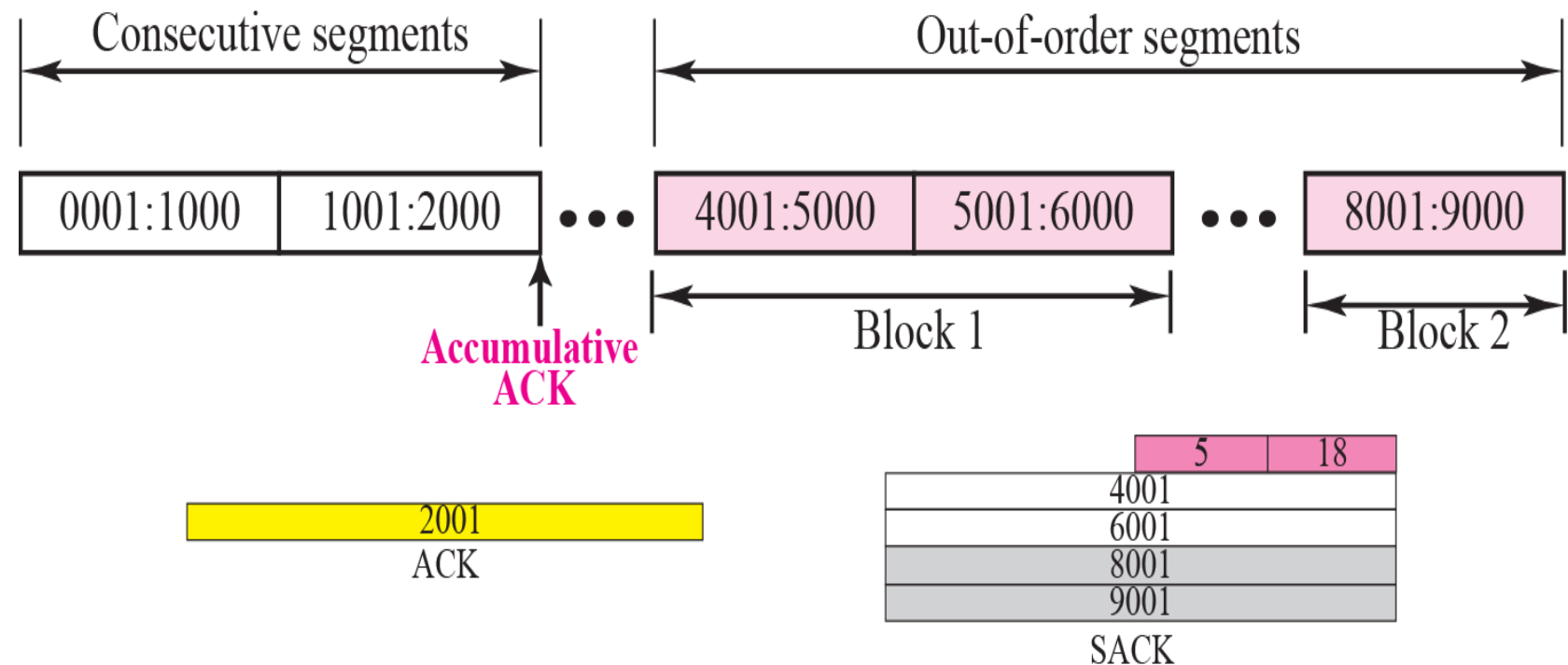
Left edge of nth Block

Right edge of nth Block

SACK option

Example

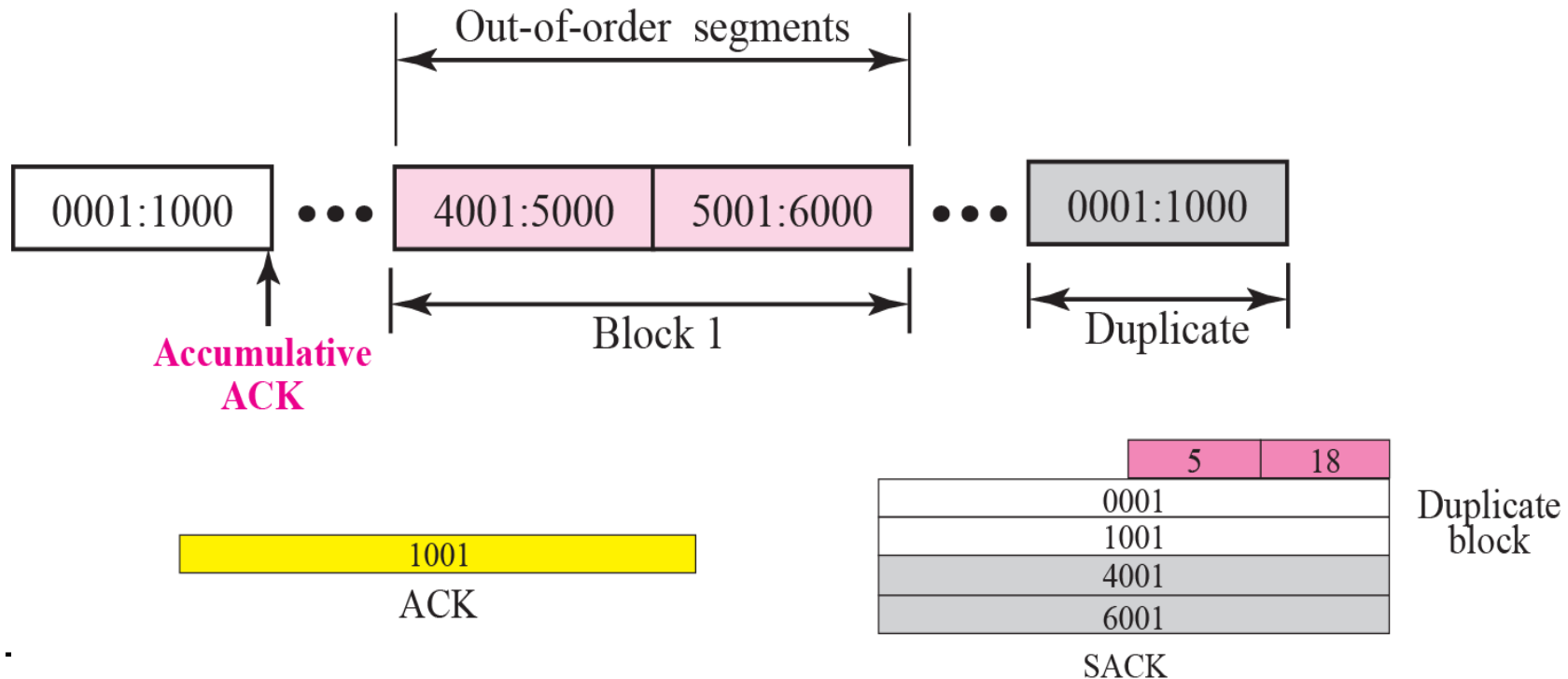
the SACK option is used to list out-of-order blocks.
In Figure one end has received five segments of data.



Example

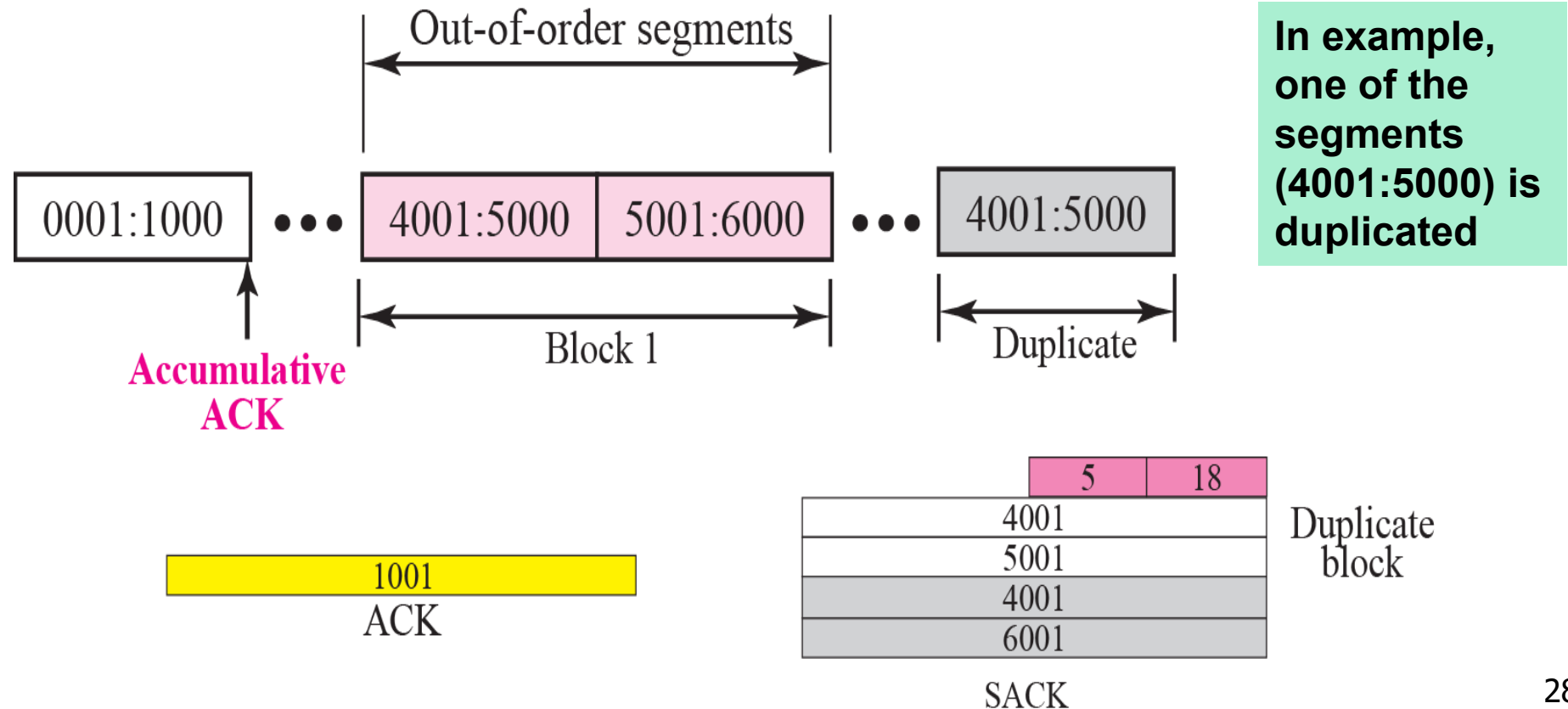
Duplicate segment can be detected with a combination of ACK and SACK.

Note that only the first block can be used for duplicate data.



Example

what happens if one of the segments in the out-of-order section is also duplicated?



TCP connection has been established and client has declared its window size as 20000 bytes. Server already sent few segments to client.

- ✓ A third and fifth segment has sequence number 14600 and 19921 respectively.
 - ✓ After sending fifth segment, server gets the third duplicate ACK from the client with the ACK no.17001.
 - ✓ At this point, what will be the threshold value (in bytes) and
 - ✓ also determine the size of the fourth segment.
- ⇒ [Assuming the Congestion window is also set to the same size as the receiver window]

Segmen	Seq number	Data bytes	Size
1			
2			
3	14600	? 14600 - 17000	
4	? 17001	? 17001 - 19920	? 2920 bytes
5	19921		

TCP connection has been established and client has declared its window size as 20000 bytes. Server already sent few segments to client. A third and fifth segment has sequence number 1460 and 19921 respectively. After sending fifth segment, server gets the third duplicate ACK from the client with the ACK no.17001. At this point, what will be the threshold value (in bytes) and also determine the size of the fourth segment.

⇒ [Assuming the Congestion window is also set to the same size as the receiver window]

Solution

Window size (rwnd) = 20000 bytes

Congestion window (cwnd) = 20000 bytes

From server to client

	Segment no	Seq number	Data bytes	Size
After sending fifth segment, server gets the third duplicate ACK from the client with the ACK no.17001 It means the next byte expected is 17001.	1			
	2			
	3	14600	? 14600 - 17000	
	4	? 17001	? 17001 - 19920	? 2920 bytes
	5	19921		

Also 17001 lies between 1460 & 19921, therefore, 4th segment sequence number will be 17001

3rd segment = 1460 – 17000

4th segment = 17001 – 19920

After 3rd duplicate, retransmission occurs and threshold will be evaluated as follows

Threshold = $\frac{1}{2} * 20000 = 10,000$ bytes

Size of fourth segment = 19920 – 17001 = 2920 bytes

Ques :

Show by drawing a clear figure for the following scenario using a TCP acknowledgement policy.

a. The sender sends three packets. The first and second packets arrived and acknowledged. The third packet is delayed and resent. The duplicate packet is received after the acknowledgment for the original is sent.

Ques :

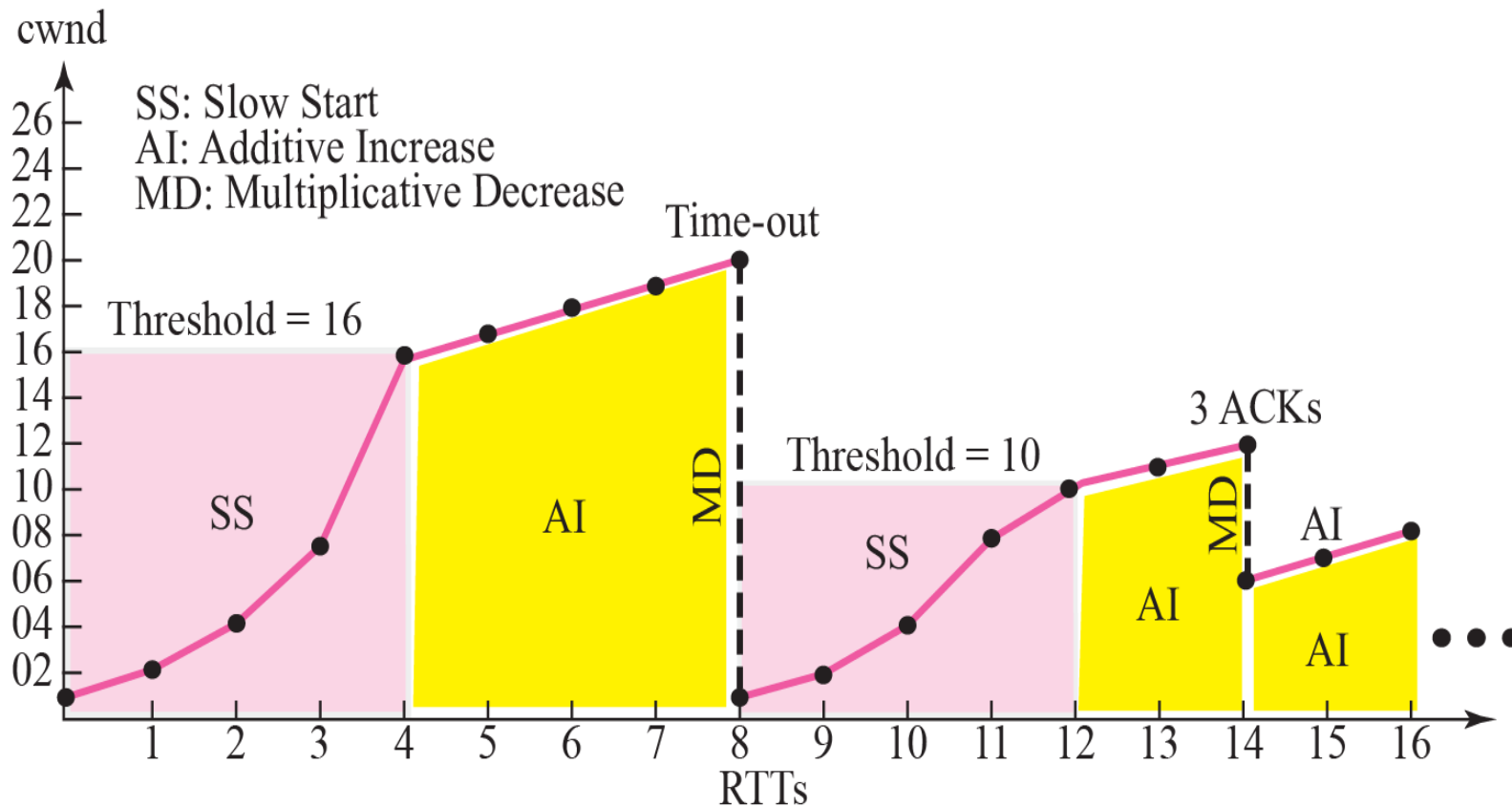
Show by drawing a clear figure for the following scenario using a TCP acknowledgement policy.

b. The sender sends two packets. The first packet is received and acknowledged, but the acknowledgement is lost. The sender resends the packet after time-out. The second packet is lost and resent.

Ques :

Show by drawing a clear figure for the following scenario using a TCP acknowledgement policy.

c. The sender sends 5 packets (0, 1, 2, 3, and 4). Packets 0, 1, and 2 are received in order and acknowledged, one by one. Packet 3 is delayed and received after packet 4.



1. Revise the transition assuming the retransmission happens at RTT 10 (3 Ack) and then at RTT 13 (Time-out)
2. What is the rwnd at start assuming the segment size is 1000 B ?

Lecture Videos on this topic can be found at following links

Transmission Control Protocol - TCP Part-1

<https://youtu.be/7tQBgArvvMA>

TCP-Part2 - TCP Connection

<https://youtu.be/5FsrWyzXskw>

TCP-Part3 - Windows in TCP and Flow Control

<https://youtu.be/Oc5OiScAlzg>

TCP Flow Control

<https://youtu.be/t7jkz9M1dHk>

Silly Window Syndrome

<https://youtu.be/-8YJBXPTi4I>

TCP - Error Control Mechanism

<https://youtu.be/gwKPjmrICJE>

TCP transmission – TCP Acknowledgement Policy

<https://youtu.be/9hIFo1F-AXU>

Lecture Videos on this topic can be found at following links

TCP Congestion control

https://www.youtube.com/watch?v=iNmV_QBP3wo

TCP Congestion Policy and Example

<https://youtu.be/-6RQvjkkkgMA>

TCP Timers

<https://youtu.be/sDWFlniQh6o>

TCP Sequence Number and Wrap Around Time (WAT)

<https://youtu.be/xTyOvYDXEHs>

Karn's Algorithm

<https://youtu.be/KlzHXu3CUIYk>

RTO Calculations -

<https://youtu.be/nYyniXX5h5o>

TCP Header OPTION

<https://youtu.be/C28IDraZ2zs>